MICROPROGRAMMING
ORGANISASI DAN ARSITEKTUR KOMPUTER

Dosen Pengampu:
Yudi Sutanto, S.Kom., M.Kom.

Kelas: IF04
Kelompok 7

Ita Alfira Lala	(24.11.6127)
Adlia Syifa Zahara	(24.11.6146)
Ervila Nirmala	(24.11.6152)
Naura As-sheiha virdina Alyan	(24.11.6165)
Keisha Leana Allung	(24.11.6166)

S1-INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS AMIKOM YOGYAKARTA

2025/2026

BAB I
PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi komputer yang semakin pesat menuntut sistem pemrosesan yang mampu bekerja lebih cepat, efisien, dan fleksibel. Salah satu komponen penting yang berperan besar dalam hal tersebut adalah pemrograman mikro (microprogramming). Pemrograman mikro merupakan teknik pengendalian operasi internal CPU melalui sekumpulan instruksi tingkat rendah yang disebut instruksi mikro, yang mengatur bagaimana setiap instruksi mesin dijalankan oleh perangkat keras.

Keberadaan pemrograman mikro sangat berpengaruh terhadap desain dan kinerja arsitektur komputer. Dengan pengendalian yang lebih terstruktur, pemrograman mikro memungkinkan CPU mengeksekusi instruksi secara lebih sistematis, mengurangi kompleksitas perancangan perangkat keras, serta meningkatkan efisiensi pemrosesan instruksi. Oleh karena itu, pemahaman mengenai konsep pemrograman mikro menjadi hal yang penting bagi mahasiswa dan praktisi di bidang teknologi informasi dan sistem komputer.

1.2. Tujuan Laporan

Laporan ini disusun dengan tujuan:

- Memberikan pemahaman mendalam
- Menganalisis peran microprogramming
- Mengevaluasi aplikasi praktis

1.3. Ruang Lingkup

Ruang lingkup pembahasan dalam laporan ini meliputi sejarah perkembangan pemrograman mikro, konsep dasar dan prinsip kerjanya, struktur serta fungsi instruksi mikro, serta implementasi dan penggunaannya pada sistem TI8800 sebagai contoh penerapan microprogramming dalam lingkungan pembelajaran dan simulasi komputer.

BAB II

KONSEP DASAR

2.1. Pengertian Microprogramming

Secara Pemrograman Mikro bukan sekadar teknik pemrograman dalam arti konvensional, melainkan sebuah metode sistematis untuk mengimplementasikan logika kontrol pada prosesor. Jika kita melihat hierarki komputer, mikro program berada pada Level 0 (Microarchitectural Level). Secara teknis, mikro program adalah sekumpulan baris kode yang disebut Control Words. Setiap Control Word berisi bit-bit biner yang secara langsung terhubung ke jalur kendali (control lines) pada komponen fisik seperti ALU, Register, dan Bus. Dengan demikian, microprogramming adalah cara "memprogram" perangkat keras agar berperilaku sesuai dengan instruksi bahasa mesin (Machine Code) yang diberikan oleh pengguna.

Konsep ini memecah sebuah instruksi makro menjadi rangkaian Micro-operations. Sebagai contoh, instruksi ADD R1, R2 dalam bahasa mesin sebenarnya terdiri dari beberapa langkah mikro:

1. Mengambil alamat dari PC (Program Counter).
2. Mengambil instruksi dari memori ke IR (Instruction Register).
3. Mengirimkan isi R1 dan R2 ke input ALU.
4. Mengaktifkan sinyal kontrol 'ADD' pada ALU.
5. Menyimpan hasil kembali ke R1.

2.2. Fungsi Microprogramming

Dalam perspektif arsitektur, tanpa fungsi microprogramming, unit kontrol harus dibangun menggunakan logika *hardwired* yang sangat kaku dan sulit dimodifikasi. Microprogramming memberikan elemen intelegensi perangkat keras yang memungkinkan prosesor menjadi lebih fleksibel, terstruktur, dan mampu menangani instruksi-instruksi kompleks secara efisien.

2.2.1. Fungsi Abstraksi Instruksi (Mapping dan Decoding)

Fungsi utama microprogramming adalah sebagai mekanisme interpretasi internal. Setiap instruksi bahasa mesin (seperti ADD, LOAD, atau JMP) hanyalah kode biner abstrak bagi perangkat keras. Microprogramming menerjemahkan satu instruksi makro menjadi serangkaian langkah atomik yang disebut operasi mikro. Ketika instruksi masuk ke Instruction Register (IR), bagian opcode akan dipetakan ke alamat tertentu dalam Control Store yang berisi rutinitas mikro yang sesuai untuk instruksi tersebut.

2.2.2. Fungsi Pengendalian Jalur Data (Datapath Control)

Microprogramming berfungsi sebagai "konduktor" yang mengatur aliran data melalui jalur fisik di dalam CPU. Hal ini dilakukan melalui sinyal-sinyal kendali yang dikirimkan ke datapath.Aktivasi Gerbang akan menentukan kapan sebuah

register harus melepaskan datanya ke bus dan kapan register tujuan harus menangkap data tersebut. Pada konfigurasi komponen mengirimkan sinyal ke ALU untuk menentukan jenis operasi yang akan dilakukan (misalnya, memerintahkan ALU untuk melakukan penjumlahan pada satu siklus, dan pergeseran bit pada siklus berikutnya). Manajemen bus akan memastikan tidak ada dua komponen yang menggunakan bus data secara bersamaan untuk menghindari konflik elektrik (bus contention).

2.2.3. Fungsi Sinkronisasi Waktu dan Sekuensing

Dalam konsep arsitektur, waktu adalah segalanya. Microprogramming memastikan bahwa setiap tindakan perangkat keras terjadi pada pulsa detak (*clock pulse*) yang tepat. Pembangkitan Sinyal Sekuensial Program mikro mengatur urutan eksekusi langkah demi langkah. Misalnya, dalam instruksi pengambilan data, program mikro memastikan alamat dikirim ke memori terlebih dahulu sebelum sinyal "Read" diaktifkan. Micro-sequencing: Menentukan alamat instruksi mikro berikutnya. Jika suatu instruksi memerlukan percabangan (seperti *Jump if Zero*), program mikro akan memeriksa *status flags* dari ALU dan memutuskan jalur mana yang akan diambil di dalam *Control Store*.

2.2.4. Fungsi Emulasi dan Fleksibilitas Arsitektur

Salah satu fungsi paling revolusioner dari microprogramming adalah kemampuannya untuk memisahkan arsitektur (apa yang dilihat pemrogram) dari implementasi (bagaimana perangkat keras sebenarnya bekerja). Kompatibilitas iet instruksi dengan mengubah isi *Control Store*, satu perangkat keras fisik dapat menjalankan berbagai set instruksi yang berbeda (misalnya, mesin x86 yang sebenarnya menjalankan instruksi mikro bergaya RISC di dalamnya). Koreksi Bug (Microcode Updates) jika terdapat kesalahan logika dalam desain prosesor, produsen dapat memperbaikinya melalui pembaruan *microcode* yang mengubah cara instruksi tersebut dieksekusi tanpa perlu menarik kembali produk fisik.

2.2.5. Fungsi Optimalisasi Sinyal (Vertical vs Horizontal)

Microprogramming berfungsi untuk mengelola kepadatan sinyal kontrol agar penggunaan memori internal menjadi efisien. Micro-programming Horizontal Digunakan untuk memaksimalkan paralelisme dengan mengontrol banyak komponen secara simultan dalam satu kata kontrol (*control word*). Micro-programming Vertical Digunakan untuk menghemat ruang dengan mengkodekan sinyal-sinyal kontrol. Hal ini mengurangi ukuran *Control Store* namun memerlukan dekoder tambahan untuk menerjemahkan kode tersebut kembali menjadi sinyal perangkat keras.

2.3. Perbedaan Microprogramming dan Pemrograman Tradisional

2.3.1. Tingkat Abstraksi dan Target Kendali

Pemrograman tradisional beroperasi pada level aplikasi atau sistem operasi. Target kendalinya adalah sumber daya sistem secara makro, seperti variabel, struktur data, dan manajemen memori. Pemrogram tidak perlu tahu bagaimana gerbang logika bekerja untuk menjumlahkan dua angka. Sedangkan pemrograman mikro: Beroperasi pada level arsitektur mikro (*Microarchitecture*). Target kendalinya adalah sinyal-sinyal elektrik spesifik. Satu baris kode mikro menentukan apakah sebuah gerbang register terbuka, apakah ALU melakukan fungsi *add* atau *shift*, dan ke mana data mengalir di jalur bus.

2.3.2. Lokasi Penyimpanan dan Media Eksekusi

Pemrograman tradisional program disimpan di memori utama (RAM) atau penyimpanan sekunder (SSD/HDD). Instruksi ini diambil oleh CPU melalui siklus *fetch* dari luar kepingan prosesor. Sedangkan pemrograman mikro disimpan di dalam memori internal prosesor yang sangat cepat, yang disebut Control Store (biasanya berupa ROM atau SRAM khusus). Instruksi mikro tidak pernah keluar dari kepingan silikon CPU.

2.3.3. Granularitas Instruksi

Perbedaan paling mencolok terletak pada "seberapa banyak" pekerjaan yang dilakukan oleh satu instruksi. Instruksi Tingkat Tinggi (Makro): Bersifat kompleks. Contoh: Instruksi MULTIPLY A, B dalam bahasa mesin mungkin terlihat sederhana bagi pemrogram, namun bagi CPU ini adalah tugas besar. Instruksi Mikro Bersifat elementer dan atomik. Untuk menjalankan satu instruksi MULTIPLY di atas, pemrograman mikro harus memecahnya menjadi puluhan langkah kecil: (1) ambil data A, (2) masukkan ke register sementara, (3) lakukan penambahan berulang di ALU, (4) simpan hasil ke register B.

2.3.4. Determinisme dan Kecepatan Sinyal

Pemrograman tradisional kecepatan eksekusi bergantung pada manajemen memori, *cache*, dan penjadwalan sistem operasi. Waktu eksekusinya relatif (tidak selalu sama tiap kali dijalankan). Sedangkan pemrograman mikro sangat deterministik. Setiap instruksi mikro dieksekusi tepat dalam satu atau beberapa siklus detak jam (*clock cycle*) yang pasti.

2.4. Keuntungan dan Kerugian Pemrograman Mikro

Keuntungan

1. Fleksibilitas Tinggi: Modifikasi atau penambahan instruksi baru hanya perlu memperbarui isi memori kontrol (*Control Store*) tanpa mengubah sirkuit fisik.
2. Penyederhanaan Hardware: Menggantikan logika gerbang yang rumit dan semrawut (*hardwired*) dengan struktur memori yang rapi dan terorganisir.
3. Kemampuan Emulasi: Memungkinkan satu prosesor menjalankan set instruksi dari arsitektur lain hanya dengan mengganti rutin mikronya.
4. Efisiensi Biaya Pengembangan: Lebih mudah dirancang, didebug, dan dipelihara, terutama untuk prosesor dengan set instruksi kompleks (CISC).

Kerugian

1. Kecepatan Lebih Lambat: Adanya *overhead* waktu karena setiap instruksi harus mengambil rutin mikro dari memori kontrol, berbeda dengan sirkuit *hardwired* yang bekerja secara instan.
2. Kompleksitas Desain Mikro: Membutuhkan ketelitian sangat tinggi; kesalahan satu bit pada instruksi mikro dapat menyebabkan kegagalan sistem secara keseluruhan.
3. Penggunaan Area Chip: Memerlukan ruang tambahan pada silikon untuk menempatkan unit memori kontrol dan perangkat pengalamatan mikro.

BAB III
PENGURUTAN INSTRUKSI

3.1. Pengertian Microinstruction

Microinstruction adalah tingkat rendah yang berisi sinyal kontrol untuk mengatur kerja komponen internal CPU. Setiap microinstruction mengeksekusi satu langkah mikro (micro-operation). Tugas utama sequencer adalah menentukan alamat instruksi mikro berikutnya yang harus diambil dari control memory. Microinstruction berisi sinyal kontrol yang mengatur register, ALU, dan jalur data.

Peran microprogramming sangat penting:

1. Mengendali sinyal kontrol Hardware
2. Menjembatani instruksi mesin dan Hardware
3. Menentukan urutan eksekusi instruksi
4. Mendukung fleksibilitas desain CPU

3.2. Struktur Microinstruction

Struktur microinstruction terdiri dari field utama:

1. Control Field: Menentukan sinyal kontrol yang diaktifkan.
2. Condition Field: Menentukan kondisi percabangan.
3. Next Address Field: Menentukan alamat instruksi mikro berikutnya.

3.3. Kode Microprogramming

Kode mikro (microcode) merupakan sekumpulan instruksi mikro yang disimpan di dalam control memory. Kode mikro digunakan untuk mengatur sinyal kontrol internet CPU sehingga suatu instruksi mesin dapat diakses secara bertahap.

Tabel 1. Format Instruksi Mikro

Field	Nama Field	Fungsi
F1	Control Signal Field	Mengaktifkan operasi internal seperti transfer register, operasi ALU
F2	Register Select Field	Menentukan register sumber dan tujuan
F3	ALU Control Field	Menentukan jenis operasi ALU (ADD, SUB, AND, OR)
F4	Memory Control Field	Mengatur operasi memori (Read / Write)
F5	Next address Field	Menentukan alamat instruksi mikro berikutnya
F6	Condition Field	Menentukan percabangan berdasarkan kondisi (Zero, carry,)

Tabel 2. Kode Mikro untuk Instruksi Mesin

Alamat Mikro	Instruksi Mikro	Deskripsi Operasi
µ00	MAR ← PC	Memindahkan isi program counter ke memory address register
µ01	IR ← M[MAR], PC ← PC + 1	Mengambil instruksi dari memori dan menaikan PC
µ02	Decode IR	Melakukan decoding instruksi
µ03	A ← R1	Memindahkan operand dari register R1 ke register A
µ04	B ← R2	Memindahkan operand dari register R2 ke register B
µ05	A ← A + B	Operasi penjumlahan menggunakan ALU
µ06	R3 ← A	Menyimpan hasil register tujuan

Tabel 3. Tabel Mikro

Opcode Instruksi Mesin	Nama Instruksi	Alamat Awal Mikro
0001	LOAD	μ10
0010	STORE	μ20
0011	ADB	μ30
0100	SUB	μ40
0101	AND	μ50
0110	OR	μ60

Tabel 4. Pengurutan Instruksi Mikro untuk ADD

Urutan	Alamat Mikro	Instruksi Mikro	Keterangan
1	μ30	A ← R1	Mengambil operand pertama
2	μ31	B ← R2	Mengambil operand kedua
3	μ32	A ← A + B	Proses penjumlahan di ALU
4	μ33	R3 ← A	Menyimpan hasil
5	μ34	Jump Fetch	Kembali ke siklus fetch instruksi

3.3. Contoh Pengurutan

Pengurutan instruksi mikro adalah proses penentuan urutan eksekusi instruksi mikro yang dilakukan oleh control unit dalam mengakses satu instruksi mesin. Setiap instruksi mesin tidak dijalankan secara langsung oleh perangkat keras, melainkan diuraikan menjadi rangkaian instruksi mikro yang diakses secara berurutan.

3.3.1. Siklus Fetch Instruksi

Urutan	Alatmat Mikro	Instruksi Mikro	Penjelasan
1	μ00	MAR ← PC	Alamat instruksi diambil dari PC
2	μ01	MDR ← M[MAR]	Instruksi diambil dari memori
3	μ02	IR ← MDR, PC ← PC + 1	Instruksi disimpan ke IR, PC ditambah
4	μ03	Decode IR	Instruksi didekode untuk menentukan operasi

3.3.2. Siklus Eksekusi Instruksi ADD

Urutan	Alamat Mikro	Instruksi Mikro	Penjelasan
5	μ30	A ← R1	Operand pertama dipindahkan ke register A
6	μ31	B ← R2	Operand kedua dipindahkan ke register B
7	μ32	A ← A + B	ALU melakukan operasi penjumlahan
8	μ33	R3 ← A	Hasil disimpan ke register tujuan
9	μ34	Jump to μ00	Kembali ke siklus fetch

3.3.3. Diagram Alur Pengurutan Instruksi Mikro

1. CPU mengambil instruksi dari memori (fetch).
2. Instruksi didekode untuk menentukan alamat awal microprogram.
3. Microinstruction dieksekusi satu per satu:
   - Pemindahan data antar register

- Operasi ALU
- Penyimpanan hasil

4. Setelah instruksi selesai, kontrol kembali ke alamat mikro awal (fetch cycle)

3.3.4. Contoh Implementasi Microprogramming TI8800

Pada sistem TI8800, instruksi mesin memiliki alamat awal microprogram di control memory. Ketika opcode ADD terdeteksi, control unit akan:

1. Melompat ke alamat awal microprograman ADD (μ30).
2. Menjalankan instruksi mikro μ30 sampai μ34 secara berurutan.
3. Setelah selesai, kembali ke siklus pengambilan instruksi berikutnya.

BAB IV
EKSEKUSI INSTRUKSI

4.1. Proses Eksekusi Instruksi

Proses eksekusi instruksi mikro merupakan tahapan kerja CPU dalam menjalankan instruksi mesin melalui serangkain instruksi mikro yang tersimpan di dalam control memory. Instruksi mikro berfungsi untuk mengaktifkan sinyal-sinyal kontrol yang mengatur operasi fisik komponen internal CPU, seperti register, ALU, dan sistem memori.

Langkah-langkah eksekusi instruksi mikro dalam CPU adalah sebagai berikut:

1. Pengambilan instruksi Mikro (Microinstruction Fetch)
   Control Unit mengambil instruksi mikro dari control memory berdasarkan alamat yang tersimpan dalam microprogram counter. Instruksi mikro ini menjadi acuan operasi yang akan dijalankan pada satu siklus kontrol.
2. Penerjemah instruksi Mikro (Microinstruction Decode)
   Instruksi mikro yang telah diambil kemudian didekode untuk menentukan sinyal-sinyal kontrol yang harus diaktifkan. Setiap field dalam instruksi mikro akan diterjemahkan menjadi perintah spesifik bagi komponen hardware.
3. Aktivasi sinyal kontrol
   Setelah proses decoding, sinyal kontrol diaktifkan untuk menjalankan operasi fisik, seperti pemindahan data antar register, operasi aritmatika dan logika oleh ALU, atau proses baca/tulis ke memori.
4. Pelaksanaan operasi fisik
   Operasi yang telah ditentukan oleh sinyal kontrol dijalankan secara langsung oleh perangkat keras CPU. Pada tahap ini, perubahan data terjadi secara fisik di dalam register, ALU, atau memori.
5. Penentuan alamat instruksi mikro berikutnya
   Setelah satu instruksi mikro selesai dieksekusi, control unit menentukan alamat instruksi mikro selanjutnya. Penentuan ini dapat bersifat berurutan atau melalui percabangan berdasarkan kondisi tertentu, seperti hasil operasi ALU.
6. Pengulangan hingga instruksi mesin selesai
   Proses eksekusi instruksi mikro akan terus berulang hingga seluruh rangkaian instruksi mikro untuk satu instruksi mesin selesai, kemudian CPU kembali ke siklus pengambilan instruksi mesin berikutnya.

Instruksi mikro dapat diterjemahkan secara sistematis menjadi operasi fisik yang dijalankan oleh CPU. Proses tersebut memastikan setiap instruksi dengan tepat

dan terkontrol, memberikan fleksibilitas tinggi dalam desain dan pengembangan arsitektur komputer.

4.2. Hubungan Antara Instruksi

Instruksi tingkat tinggi merupakan perintah yang ditulis oleh programmer menggunakan bahasa pemrograman seperti C, C++, atau Java untuk menyelesaikan suatu tugas tertentu. Instruksi ini tidak dapat dieksekusi secara langsung oleh perangkat keras komputer, sehingga harus melalui beberapa tahap penerjemahan hingga dapat dijalankan oleh CPU.

instruksi mikro bertindak sebagai jembatan penghubung antara instruksi tingkat tinggi dan perangkat keras. Setiap instruksi tingkat tinggi dapat menghasilkan banyak instruksi mesin, dan setiap instruksi mesin dapat diuraikan lagi menjadi beberapa instruksi mikro. Hubungan ini memastikan bahwa perintah yang ditulis pada tingkat perangkat lunak dapat dieksekusi secara tepat dan efisien pada tingkat perangkat keras.

4.3. Pengaruh Eksekusi

Eksekusi instruksi mikro memiliki pengaruh langsung terhadap kecepatan dan efisiensi proses pada sistem komputer. Instruksi mesin dijalankan melalui beberapa instruksi mikro yang mengatur operasi internal CPU. Pada sistem yang menggunakan microprogramming, proses eksekusi cenderung lebih lambat dibandingkan dengan sistem hardwired control karena adanya tahapan tambahan, seperti pengambilan dan decoding instruksi mikro dari control memory.

Microprogramming dapat meningkatkan efisiensi sistem secara keseluruhan dengan mengoptimalkan urutan instruksi mikro dan mengurangi kompleksitas rangkaian kontrol. Eksekusi instruksi mikro dapat menambah waktu eksekusi, microprogramming tetap menjadi solusi yang seimbang antara kecepatan, fleksibilitas, dan kemudahan pengembangan sistem komputer.

4.3. Contoh Proses Eksekusi

Pada arsitektur komputer berbasis microprogramming, seperti TI8800. Instruksi ini bertujuan menjumlahkan isi register R1 dan R2, kemudian menyimpan hasilnya ke register R3.

4.3.1. Langkah-langkah Eksekusi Instruksi Mikro

Urutan	Alamat Mikro	Instruksi Mikro	Penjelasan
1	μ00	MAR ← PC	Mengambil alamat instruksi dari Program Counter
2	μ01	MDR ← M[MAR]	Instruksi mesin diambil dari memori
3	μ02	IR ← MDR, PC ← PC + 1	Instruksi disimpan di Instruction Register, PC dinaikkan
4	μ03	Decode IR	Instruksi ADD dikenali dan microprogram ADD dijalankan
5	μ30	A ← R1	Operand pertama dipindahkan ke register A
6	μ31	B ← R2	Operand kedua dipindahkan ke register B
7	μ32	A ← A + B	ALU menjumlahkan nilai A dan B
8	μ33	R3 ← A	Hasil penjumlahan disimpan ke register tujuan R3
9	μ34	Jump Fetch	Kembali ke siklus pengambilan instruksi berikutnya

BAB V
MICROPROGRAMMING COMPUTER

5.1. Sejarah dan Pengenalan TI8800

5.1.1. Latar Belakang Sejarah TI8800

TI8800 (sering juga disebut Altair 8800) merupakan salah satu komputer mikro paling berpengaruh dalam sejarah perkembangan teknologi komputer. Komputer ini diperkenalkan pada tahun 1975 oleh perusahaan MITS (Micro Instrumentation and Telemetry Systems) di Amerika Serikat. Pada masa itu, komputer masih berukuran besar (mainframe & minicomputer), sangat mahal, dan hanya dapat diakses oleh institusi besar seperti universitas dan perusahaan. Kehadiran TI8800 menandai awal era komputer pribadi (personal computer) karena untuk pertama kalinya komputer dapat dimiliki dan dirakit oleh individu, terutama para hobi elektronik.

5.1.2. Peranan TI8800 dalam Perkembangan Teknologi Komputer

TI8800 dianggap sebagai pemicu revolusi komputer pribadi, membuka peluang bagi masyarakat umum untuk memiliki dan mengembangkan komputer sendiri. TI8800 mendorong berkembangnya software independen. Salah satu peristiwa penting adalah Bill Gates dan Paul Allen mengembangkan BASIC interpreter untuk TI8800 dan hal ini menjadi cikal bakal berdirinya Microsoft.

5.2. Arsitektur TI8800

5.2.1. Komponen Utama dalam Arsitektur TI8800

a. Unit Pemroses Pusat (CPU)

Unit Pemroses Pusat (CPU) menggunakan Intel 8080 dengan arsitektur 8-bit dengan address bus 16-bit (maks. 64 KB memori), dan CPU ini menjadi inti dari seluruh sistem TI8800.

b. Unit Kontrol (Control Unit)

Unit Kontrol (Control Unit) mengatur urutan eksekusi instruksi, pengambilan instruksi dari memori, dan sinkronisasi kerja antar komponen. Pada TI8800, unit kontrol diimplementasikan menggunakan microprogramming, bukan kontrol kabel keras (hardwired).

c. Arithmetic Logic Unit (ALU)

Arithmetic Logic Unit (ALU) berfungsi untuk Operasi aritmatika (penjumlahan, pengurangan), Operasi logika (AND, OR, XOR), dan Operasi perbandingan. ALU bekerja di bawah kendali sinyal yang dihasilkan oleh unit kontrol mikroprogram.

d. Memori Utama

Memori Utama menggunakan RAM statik/dinamik awal, terhubung melalui bus S-100, dan kapasitas memori awal relatif kecil, tetapi dapat diperluas melalui kartu tambahan.

e. **Sistem Bus (S-100 Bus)**
   Bus S-100 berfungsi sebagai jalur data, jalur alamat, dan jalur kontrol. Bus ini memungkinkan modularitas sistem, sehingga pengguna dapat menambahkan kartu CPU, memori, atau I/O.
f. **Perangkat Input dan Output**
   Perangkat Input dan Output berupa panel depan (switch dan LED) sebagai input/output dasar, terminal eksternal (keyboard dan monitor) melalui kartu antarmuka, dan media penyimpanan seperti tape atau disk (opsional).

5.2.2. Mekanisme Microprogramming pada TI8800

Pada TI8800 (dengan CPU Intel 8080), microprogramming digunakan pada unit kontrol untuk mengatur bagaimana setiap instruksi mesin dieksekusi. Instruksi tidak dijalankan langsung oleh hardware tunggal, tetapi dipecah menjadi langkah-langkah kecil yang disebut microinstruction.

a. Penyimpanan Microprogram
   Microinstruction disimpan di dalam control memory (biasanya ROM mikro). Setiap instruksi mesin (misalnya MOV, ADD, JMP) memiliki microprogram tersendiri.
b. Tahap Eksekusi Instruksi Berbasis Microprogramming
   - Fetch Instruksi
   - Decode Instruksi
   - Eksekusi Microinstruction
   - Penyelesaian Instruksi
c. Peran Microinstruction dalam Pengendalian Sistem
   Setiap microinstruction mengatur sinyal kontrol internal (register enable, ALU control), sinyal kontrol eksternal (akses memori melalui bus S-100), dan timing operasi agar sinkron.

5.3. Implementasi Microprogramming

Pada TI8800, microprogramming tidak berdiri sebagai komponen terpisah, melainkan tertanam di dalam mikroarsitektur CPU Intel 8080 yang digunakan sebagai otak sistem. Unit kontrol CPU ini diimplementasikan menggunakan microprogram, bukan kontrol hardwired murni. Microprogramming berfungsi sebagai lapisan pengendali internal yang menerjemahkan instruksi mesin menjadi aksi-aksi hardware dasar.

Contoh implementasi mikro pada perangkat ini:

Langkah Mikro	Operasi
µ1	PC → Address Bus
µ2	Memory Read
µ3	Instruction → IR
µ4	PC = PC + 1

5.4. Kelebihan dan Kekurangan TI8800

Kelebihan	**Kekurangan**
Pelopor Komputer Pribadi	Antarmuka sangat terbatas
Menggunakan mikroprosesor	Sulit digunakan oleh pengguna umum
Arsitektur Modular (Bus S-100)	Kapasitas memori kecil
Mendorong perkembangan software	Kecepatan pemrosesan rendah
Nilai Edukatif Tinggi	Perangkat lunak terbatas

BAB VI
APLIKASI DAN PENGEMBANGAN

6.1. Penggunaan Microprogramming

6.1.1. Penggunaan Microprograming dalam Teknologi Modern

Microprogramming telah berevolusi dari sekadar metode penghematan perangkat keras di masa lalu menjadi teknik optimasi yang krusial dalam arsitektur komputer modern. Secara mendasar, microprogramming adalah teknik di mana instruksi tingkat tinggi (machine code) dipecah menjadi urutan operasi tingkat rendah yang disebut **micro-instructions**.

Berikut adalah penerapan prinsip microprogramming dalam berbagai arsitektur modern:

1. **Peran dalam Arsitektur CISC (Complex Instruction Set Computer)**

   Arsitektur CISC, seperti keluarga Intel x86, sangat bergantung pada microprogramming. Karena instruksi x86 sangat kompleks dan bervariasi panjangnya, prosesor tidak mengeksekusinya secara langsung di perangkat keras.

   - Penerjemahan (Decoding): Prosesor x86 modern sebenarnya memiliki inti yang mirip RISC. Instruksi CISC yang kompleks diterjemahkan oleh unit dekoder menjadi serangkaian Micro Operation (uops).
   - Microcode ROM: Instruksi-instruksi ini disimpan dalam memori internal kecil di dalam CPU.
   - Fleksibilitas Patching: Keuntungan utama di sini adalah Microcode Updates. Produsen seperti Intel atau AMD dapat memperbaiki bug logika CPU / kerentanan keamanan seperti Spectre/Meltdown melalui pembaruan perangkat lunak yang memperbarui microcode tanpa harus mengganti fisik CPU.

2. **Implementasi pada Arsitektur RISC (Reduced Instruction Set Computer)**

   Secara tradisional, RISC dikenal dengan hardwired control (logika sirkuit langsung) untuk mengejar kecepatan. Namun, batas antara RISC dan CISC kini semakin kabur.

- Instruksi khusus: Beberapa prosesor RISC modern menggunakan microcode untuk menangani instruksi yang jarang digunakan atau sangat kompleks seperti, operasi floating-point yang rumit atau instruksi manajemen memori.
- Abstraksi Hardware: Microprogramming memberikan lapisan abstraksi yang memungkinkan desainer chip untuk mengubah implementasi internal tanpa mengubah Instruction Set Architecture (ISA) yang terlihat oleh programmer.

3. Sistem Embedded dan Pengontrol Mikro

Dalam dunia sistem tertanam (embedded Systems), microprogramming memberikan efisiensi ruang dan fungsionalitas khusus:

- Custom Instruction: Pada perangkat seperti FPGA atau prosesor khusus sinyal (DSP) microprogramming memungkinkan pembuatan instruksi custom yang dioptimalkan untuk tugas spesifik misalnya, enkripsi atau pemrosesan video.
- Emulasi Arsitektur: Microprogramming sering digunakan untuk membuat satu set perangkat keras maupun mensimulasi taua mengemulasi arsitektur proses lain, yang sangat berguna dalam pengembangan perangkat lintas platform.

6.2. Contoh Implementasi Microprogramming Sistem Embedded

Dalam sistem embedded, microprogramming seringkali menjadi jembatan antara kebutuhan akan efisiensi perangkat keras (biaya rendah dan konsumsi daya kecil) dengan kebutuhan fungsionalitas yang fleksibel.

Berikut adalah beberapa contoh penggunaan microprogramming pada perangkat elektronik atau IoT:

6.2.1. Digital Signal Processors (DSP pada perangkat Arduino

DSP adalah komponen kunci dalam perangkat seperti smart speakers atau headphone noise cancelling.

- **Implementasi**: Merancang sirkuit fisik untuk setiap jenis filter audio misalnya low pass atau equalizer, desainer menggunakan unit eksekusi mikro yang dapat diprogram.
- **Fungsi Microcode**: Bertugas mengatur urutan operasi perkalian dan akumulasi MAC Operations yang sangat cepat. Dengan mengubah microcode, produsen dapat memperbarui algoritma pemrosesan suara tanpa mengganti chip nya.

6.2.2. Pengendalian Mesin Cuci dan Alat Rumah Tangga Pintar

Banyak mikrokontroler (*microcontroller*) dalam alat rumah tangga menggunakan prinsip microprogramming untuk mengelola logika kontrol yang kompleks namun repetitif.

- **Implementasi:** Mesin cuci memiliki berbagai siklus (cuci, bilas, peras). Logika utama mesin disimpan dalam bentuk instruksi tingkat tinggi, namun urutan pembukaan katup air, putaran motor, dan aktivasi pompa dikelola oleh **microsequencer**.
- **Keuntungan:** Jika ada perubahan spesifikasi mesin (misalnya mode hemat air baru), pengembang cukup memperbarui tabel microcode di dalam memori kontrol tanpa mendesain ulang sirkuit logika pengendali.

6.2.3. Modul Komunikasi IoT (Wifi/Bluetooth)

Perangkat IoT sering menggunakan chip komunikasi khusus (seperti seri ESP32 atau modul Bluetooth Low Energy) yang memiliki kontroler internal berbasis microcode.

- **Penanganan Protokol:** Protokol nirkabel seperti Wi-Fi memiliki aturan waktu yang sangat ketat (*timing-sensitive*).
- **Fungsi Microcode:** Microprogramming digunakan untuk menangani paket data pada lapisan fisik (PHY). Jika ada standar keamanan baru (seperti transisi dari WPA2 ke WPA3), seringkali hal ini dapat diterapkan melalui **firmware/microcode update** tanpa memerlukan perangkat keras baru

6.2.4. Pengendali Unit Injeksi Bahan Bakar (ECU) Otomotif

Dalam sistem embedded otomotif, presisi adalah segalanya.

- **Implementasi:** ECU (Engine Control Unit) harus menghitung waktu pengapian dan semprotan bahan bakar berdasarkan input sensor dalam hitungan milidetik.
- **Fungsi Microcode:** Microprogramming memungkinkan ECU menjalankan rutinitas matematika kompleks (seperti interpolasi tabel peta bahan bakar) secara paralel atau dalam urutan mikro

yang sangat cepat untuk memastikan mesin tetap efisien pada berbagai suhu dan tekanan.

6.3. Perkembangan Teknologi yang Mempengaruhi Microprogramming

Perkembangan teknologi telah membawa konsep microprogramming keluar dari sekadar metode internal CPU dan masuk ke ranah sistem yang lebih luas dan fleksibel. Di era modern, prinsip ini digunakan untuk mencapai personalisasi perangkat keras dan efisiensi dalam skala besar.

Berikut adalah teknologi terkini yang menerapkan prinsip microprogramming:

6.3.1. FPGA (Field Programmable Gate Array)

FPGA adalah perwujudan paling dinamis dari microprogramming di tingkat perangkat keras. Jika pada CPU tradisional microcode bersifat tetap atau hanya bisa diperbarui secara terbatas, FPGA memungkinkan seluruh "sirkuit" diprogram ulang.

- Implementasi: FPGA menggunakan Look-Up Tables (LUT) dan bitstream yang bertindak seperti microcode raksasa. Bitstream ini menentukan bagaimana gerbang logika dihubungkan.
- Akselerasi AI: Banyak perusahaan menggunakan FPGA untuk menjalankan model AI (seperti enkripsi atau pemrosesan gambar) dengan memprogram instruksi mikro yang sangat spesifik untuk tugas tersebut, sehingga jauh lebih cepat daripada CPU standar.

6.3.2. Smart NICS dan Arsitektur Distributed System

Dalam pusat data modern dan sistem terdistribusi, microprogramming diterapkan pada kartu jaringan cerdas (SmartNICs) untuk menangani beban kerja yang biasanya membebani CPU utama.

- Offloading Tugas: Smart NICS memiliki prosesor kecil di dalamnya yang menjalankan microcode khusus untuk memproses paket data, enkripsi, dan manajemen lalu lintas jaringan secara langsung di hardware.
- Efisiensi Cloud: Penyedia layanan cloud seperti AWS atau Azure menggunakan teknik ini agar server mereka dapat fokus menjalankan aplikasi pengguna, sementara tugas-tugas administratif jaringan dikelola oleh unit kontrol yang terprogram di SmartNIC.

6.3.3. Software Defined Radio (SDR)

Teknologi komunikasi modern, terutama dalam pengembangan 5G dan satelit, sangat bergantung pada fleksibilitas microprogramming.

- **Fleksibilitas Frekuensi:** Pada SDR, fungsi-fungsi fisik radio (seperti modulasi dan penyaringan sinyal) tidak dilakukan oleh kabel/sirkuit tetap, melainkan oleh kode mikro yang berjalan pada prosesor sinyal digital (DSP) atau FPGA.
- **Adaptasi Standar:** Hal ini memungkinkan satu perangkat keras yang sama untuk mendukung berbagai standar komunikasi hanya dengan memperbarui software nya.

6.3.4. Evolusi Microcode dalam Custom Computing

Saat ini terdapat tren menuju Custom Computing Machines (mesin komputasi khusus) di mana batas antara arsitektur dan implementasi semakin kabur.

- **Arsitektur Heterogen:** Sistem modern sering menggabungkan CPU, GPU, dan akselerator khusus. Microprogramming digunakan untuk mengkoordinasikan instruksi di antara komponen-komponen yang berbeda ini agar bekerja secara sinkron tanpa latensi tinggi.
- **Virtualisasi Hardware:** Microcode memungkinkan pembuatan "mesin virtual" di tingkat perangkat keras yang dapat mensimulasikan berbagai jenis prosesor dengan efisiensi tinggi.

6.4. Tren dan Inovasi

Microprogramming sebagai teknik pengendalian CPU terus berkembang seiring dengan kemajuan teknologi prosesor dan sistem komputer modern. Meskipun konsep dasarnya sudah lama, inovasi terbarumemungkinkan penerapan microprograman pada arsitektur yang lebih kompleks dan fleksibel, termasuk prosesor multi-core, sistem embedded, dan perangkat reconfigurable.

6.4.1. Microprogramming pada Prosesor Multi-Core

Penggunaan microprogramming pada prosesor multi-core memungkinkan setiap inti (core) prosesor memiliki microcode yang dioptimalkan untuk eksekusi instruksi spesifik. Hal ini mendukung:

- Distribusi beban kerja (workload distribution) antar core secara efesien.
- Eksekusi instruksi kompleks dengan koordinasi antar core.
- Penggunaan microcode yang dapat di-update untuk memperbaiki atau menambah instruksi tanpa mengubah hardware inti.

6.4.2. Microprogramming dalam Sistem Embedded

Pada sistem embedded, microprogramming digunakan untuk:

- Mengendalikan perangkat keras dengan efisiensi tinggi dan konsumsi daya rendah.
- Memungkinkan update instruksi atau algoritma kontrol tanpa mengganti perangkat keras.
- Mendukung integrasi dengan sistem IoT, sensor, dan aktuator yang memerlukan kontrol presisi.

6.4.3. Microprogramming pada FPGA dan Sistem Reconfigurable

FPGA (*Field-Programmable Gate Array*) memungkinkan implementasi microprogramming secara dinamis:

- Arsitektur FPGA dapat dikonfigurasi ulang sesuai microinstruction baru.
- Sistem reconfigurable dapat menyesuaikan operasi CPU sesuai kebutuhan aplikasi.
- Mendukung pengembangan prototipe prosesor atau sistem khusus (*custom processor design*).

6.4.4. Optimisasi Performa melalui Microcode

Inovasi terbaru juga mencakup optimisasi microcode untuk meningkatkan kecepatan eksekusi:

- Penggunaan parallel microinstructions untuk menjalankan beberapa operasi secara bersamaan.
- Pengurangan siklus microinstruction untuk instruksi yang sering digunakan.
- Implementasi cache microcode berkecepatan tinggi untuk prosesor modern.

6.4.5. Integrasi dengan Teknologi AI dan Machine Learning

Beberapa inovasi mengintegrasikan microprogramming dengan akselerator AI:

- Microcode digunakan untuk mengontrol unit pemrosesan AI khusus di dalam CPU atau GPU.
- Menyediakan fleksibilitas dalam menjalankan model ML tanpa mengubah hardware dasar.
- Memungkinkan update instruksi untuk algoritma AI terbaru secara efisien.

6.4.6. Tren Masa Depan

Beberapa tren yang diprediksi akan berkembang di masa depan:

- Microprogramming adaptif untuk prosesor heterogen dan sistem multi-core.
- Implementasi microcode pada quantum-inspired processors.
- Integrasi dengan sistem cloud dan edge computing untuk fleksibilitas kontrol hardware.
- Peningkatan keamanan melalui microcode yang dapat mencegah eksploitasi atau kesalahan eksekusi instruksi.

VII
KESIMPULAN

Microprogramming merupakan konsep penting dalam arsitektur komputer yang berfungsi sebagai mekanisme pengendalian internal CPU. Melalui microprogramming, setiap instruksi mesin dieksekusi tidak secara langsung oleh perangkat keras, melainkan diuraikan menjadi serangkaian instruksi mikro (microinstruction) yang mengatur operasi dasar seperti pemindahan data antar register, operasi aritmatika dan logika pada ALU, serta akses memori. Hal ini menjadikan proses eksekusi instruksi lebih terstruktur dan mudah dikendalikan.

Instruksi mikro berperan sebagai unit kendali terkecil yang menjembatani instruksi tingkat tinggi dengan operasi fisik pada hardware. Dengan adanya pengurutan instruksi mikro yang tersusun dalam kode mikro dan tabel mikro, CPU dapat menjalankan berbagai instruksi mesin secara sistematis dan konsisten. Pendekatan ini memberikan fleksibilitas dalam desain prosesor serta memudahkan modifikasi dan pengembangan instruksi tanpa harus melakukan perubahan pada perangkat keras.

Penerapan microprogramming pada Micro-programmable Computer Card TI8800 menunjukkan implementasi nyata dari konsep pemrograman mikro. TI8800 berfungsi sebagai sarana pembelajaran yang efektif untuk memahami arsitektur komputer dan mekanisme kerja CPU secara mendalam. Melalui TI8800, konsep microinstruction, pengendalian eksekusi instruksi, serta hubungan antara software dan hardware dapat dipelajari secara praktis.

Secara keseluruhan, microprogramming tetap relevan dalam perkembangan teknologi komputer modern. Meskipun beberapa sistem menggunakan pendekatan pengendalian berbasis hardware untuk meningkatkan kecepatan, microprogramming masih digunakan dalam bentuk microcode pada prosesor modern, sistem embedded, serta teknologi reconfigurable seperti FPGA. Dengan fleksibilitas dan kemampuannya dalam mendukung pembaruan sistem, microprogramming akan tetap menjadi pondasi penting dalam pengembangan arsitektur komputer di masa depan.

DAFTAR PUSTAKA

Stallings, W.. “Microprogramming and system architecture.” *The Computer Journal*, vol. 17, no. 1, 01 January 1974, pp. 2-8. *OXFORD ACADEMIC*

Mick, J., *& Brick, J. (1980). Bit-slice microprocessor design. McGraw-Hill.*

Patterson, D. A., & Hennessy, J. L. (2021). Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.

Mano, M. M. (1992). Computer System Architecture. 3rd Edition. New Jersey: Prentice-Hall.

Syahrul. (2010). Organisasi dan Arsitektur Komputer. Yogyakarta: Andi Publisher.

Hwang, K., Briggs, F. A. “Computer Architecture and Microprogramming.” *IEEE Transactions on Computers*, vol. C-28, no. 12, December 1979, pp. 888–897. IEEE.

Patterson, D. A., Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*. 5th Edition, Morgan Kaufmann, 2013.

Flynn, M. J. *Computer Architecture: Pipelined and Parallel Processor Design*. 2nd Edition, Jones & Bartlett, 1995.

Hamacher, C., Vranesic, Z., Zaky, S., & Manjikian, N**.** (2012). *Computer Organization and Embedded Systems* (6th ed.). McGraw-Hill Education.

Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

Null, L., & Lobur, J. (2018). *The Essentials of Computer Organization and Architecture* (5th ed.). Jones & Bartlett Learning.

Stallings, W. (2015). *Computer Organization and Architecture: Designing for Performance* (10th ed.). Pearson.

Tanenbaum, A. S., & Austin, T. (2012). *Structured Computer Organization* (6th ed.). Pearson.

Wolf, M. (2017). *Computers as Components: Principles of Embedded Computing System Design* (4th ed.). Morgan Kaufmann.

Leupers, R., Marwedel, P. *Microcontroller-Based System Design: Fundamentals, Implementation, and Applications*. Springer, 2010.